

1 Setup

Task and data. A forecaster LLM predicts whether an AI model solves a benchmark task: for each pair (t, m) of task t and model m , it reports percentiles $p_{25}/p_{50}/p_{75}$ of $\Pr[m \text{ solves } t \text{ in one attempt}]$. Ground truth $y_{t,m} \in \{0, 1\}$ is the recorded pass/fail from the Lyptus suite of cybersecurity benchmarks (269 usable tasks, 11-model panel, tasks grouped into 5 difficulty bins by human first-solve time). The forecaster is Claude Sonnet 4.6 at temperature 0, one call per pair. Its prompt contains example tasks with outcomes and group pass rates, all drawn from difficulty bins *other than* the target’s; the target’s own difficulty bin is hidden.

Splits. Fixed once before any experiment. Search set: 84 pairs, used to select prompts during search. Held-out set: 230 pairs (21 unseen tasks $\times$ 11 models), used only for final scoring. Two whole benchmarks stay reserved for a single future out-of-distribution test.

Scoring. Accuracy is the Brier score on p_{50} , with the CRPS of a Beta fitted to the percentile triple as a secondary check; lower is better. A prompt’s score is its mean over the 230 held-out pairs; its *gain* is the paired mean difference against the seed on identical pairs in the same measurement session. Re-running a fixed prompt gives sd 0.0065 on one search-set pass and ≈ 0.002 on a multi-pass held-out estimate, so single search-set passes cannot rank prompts.

Baselines. (i) The *seed*: a minimal hand-written prompt holding the task statement, the evidence block, and the output format, with no forecasting guidance. Held-out Brier 0.1312 (8 passes). (ii) The *difficulty-oracle table*: no LLM, predict the leave-one-out mean pass rate of model m over the other tasks in the target’s difficulty bin $B(t)$,

$$b(t, m) = \frac{1}{|B(t)| - 1} \sum_{t' \in B(t), t' \neq t} y_{t', m} = 0.1034 \text{ on the held-out set.}$$

The table reads the difficulty label that is hidden from the forecaster, so it is an oracle-informed reference rather than a fair competitor.

Prompt search. GEPA (arXiv:2507.19457), with these choices: the only evolving component is the instruction text; the rewriter is Claude Opus 4.8, shown the parent prompt with its scored outputs on a fresh 20-pair batch; a proposal joins the pool if it beats its parent on that batch; one run makes 40 proposals, about 4,000 forecaster calls. Given the noise floor, final ranking always uses ≥ 3 paired held-out passes: in run 2 the search set’s first-ranked prompt had no held-out gain, while its fourth-ranked one was the real winner.

2 The prompts

Four searches produced 82 evolved prompts; with the seed and two hand-built probes (§3), 85 prompts total.

source	prompts	best held-out gain	note
seed (hand-written)	1	reference	no forecasting guidance
search run 1	25	+0.027* (8/8 paired passes)	ran with a scoring bug: unparsable answers scored as maximally wrong; its three winners re-verified after the fix
search run 2	25	+0.016 (3/3)	bug fixed, otherwise identical; independent replication
search variants	32	none beat the seed	wider acceptance test; and a rewriter told to avoid numeric content, which serves as a control in §3
single-feature probes (§3)	2	+0.020 / +0.006	seed plus exactly one feature

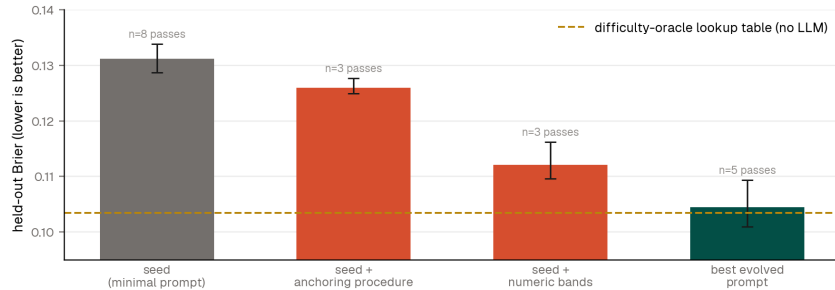
*the best evolved prompt; rightmost bar in the figure.

3 Which prompt features matter

Every evolved prompt was tagged for five recurring features with a fixed keyword rubric. Features arrive together in the same prompts, so the tag screen is directional evidence only; the probe experiment below is the causal test.

feature	what the prompt text says	n/82	Δ Brier	held-out evidence
anchoring instruction	base the probability on the observed pass rates in the evidence	82	—	in every evolved prompt
nearest-example weighting	if one example closely matches the target, weight its outcome above its group average	31	−0.0043	in all winners; absent from the no-numbers control
numeric probability bands	explicit ranges per task kind: trivial lookup 0.92–0.99, live exploitation 0.10–0.30	45	−0.0008	in all winners; absent from the control; probed below
anti-hedging	do not default to 0.5 when unsure	46	−0.0025	also present in the control’s null prompts
task-type adjustment	shift down for tasks against a live service, up for static file analysis	69	−0.0029	near-universal, including nulls

Causal probe. The best evolved prompt separates into numeric content (the probability bands) and a forecasting procedure (anchoring, nearest-example weighting, task-type adjustment, anti-hedging, written without any numbers). Each half was added to the seed alone and measured on the held-out set, 3 paired passes each. The bands alone recover most of the full prompt’s gain (+0.020 of +0.027); the procedure alone recovers a quarter (+0.006). CRPS gives the same ordering (seed 0.216, bands 0.189, best evolved 0.178).



4 Conclusions

1. Numeric anchors in the prompt do the work. Adding the winner’s numbers to the seed recovers about three quarters of its gain; adding the same procedure with the numbers removed recovers a quarter. This agrees with the spring elicitation studies, where model outputs followed whatever number the prompt supplied and removing anchors mainly increased variance. For this task, prompt design reduces to choosing which numbers the model sees; instructions that tell the model how to derive its own numbers are a weak substitute.

2. A lookup table bounds what optimization buys. The search improved the forecaster in two independent runs, in both cases by improving discrimination rather than calibration. The best evolved prompt matches the difficulty-oracle table and does not beat it, and the strongest base model tested in the spring reached the same level from a hand-written prompt. Within the training distribution the table is the practical ceiling, and optimization recovered about one model generation of accuracy.

3. The open question is transfer. A lookup table cannot extrapolate to unseen benchmarks. Whether the evolved prompt can is the remaining question, and the reserved benchmarks exist for that single test.

Reproducibility. Scripts `feature_report_data.py`, `val_noise_study.py`, `parse_failure_audit.py` (gepa repo); raw runs on `results/*` branches; curated data, prompt texts, this document: `experiments/III_gepa_optimization/summary/`; earlier stages: `prompt_sensitivity/`, experiments I/II, final report §3. 2026-08-30.